ARQUITECTURAS E INFRAESTRUCTURAS PARA INTELIGENCIA ARTIFICIAL

DOCUMENTATION: SERVERLESS CLOUD COMPUTING MODEL

Introduction:

Serverless applications are deployed regardless of the infrastructure on which they run. They are defined by code functions that are executed when the defined trigger is triggered.

Steps:

The steps to create a Serverless execution application on MS Azure are as follows:

1) Create a Function App

We are looking for the "Function Application" service
Click on "Create".
We select the "Consumption" hosting option.
Click on "Select".

Project Details:

- Subscription: Azure for Students
- Resource group: We select one ("grupoest")
- Instance details
    - Feature app name: Enter a name
    - Do you want to deploy a container code or image?

      Functions can be created by deploying the code of the functions directly or by using a Docker container prepared to handle requests. In this case we select code.

    - Runtime environment stack: We choose the language in which we want to execute the function
    - Version: Version of the language used. The most current version will generally be used.
    - Region: We will try to choose the one closest to the customers. In our case "West Europe"
- Operating system: Only Linux is available
- Click on "Next: Storage"
    - We leave the default option to create a new storage account that saves the application code
- Click on "Next: Networks"
    - Enable public access: If enabled, the feature is accessible from the internet. If it is deactivated, it is only accessible from the internal Azure network, that is, only other resources created in Azure will be able to access. Select "On"
- Click on Next: Supervision
    - Application Insights
        - Habilitar Application Insights: No
- Click on "Next: Implementation"

    - GitHub Actions settings
        - Continuous Deployment: No.

          If you have a GitHub repository with your code, you can have the function's code automatically update in Azure when you push.

- Click on "Next: Tags"
    - Assign the label and the corresponding value
- Click on "Next: Review and Create"
- Once validated, click on "Create"

2) Activate a function app

Within a function application, we can create the functions we need. These features can have different trigger criteria, and are independent of each other. Let's create a function that fires when it receives a get-type request.

We are looking for the "Function Application" service

- Click on the application we have just created

  On this screen we can see all the options of the function application.

- In the "Functions" section click on the "Functions" menu
- Click on "Create".
- Select a template

  It allows you to choose which trigger you will use to execute the function. Depending on the trigger we choose, the function will be executed when a certain event occurs. In this case we choose "HTTP trigger", which will execute the function when an HTTP request is received. Other types of triggers include, for example, "Azure Blob Storage Trigger," which allows you to execute a function when files are added to a storage container.

- Template details
    - Function Name: We choose the function name
    - Authorization level: Elegimos Function.
- Click on "Create"

  You can review the default code for the template in the "Code & Test" menu. The default code is capable of receiving a request and responding.

To test the function we enter the following url in the browser:
https://<your-function-app>.azurewebsites.net/api/<your-function>?code=<your-function-key>
We can easily access this URL by clicking on "Get the URL for the function" in the "Code and test" section.

Although this feature was developed using the web editor available in Azure, we recommend developing the features using Visual Studio Code.

In the following URL you can find a tutorial on how to use VS Code: https://learn.microsoft.com/en-us/azure/azure-functions/functions-develop-vs-code?tabs=csharp

One aspect to note is that the Serverless functions have a RAM memory limit of 1.5GB in their default consumption plan, which limits the use of these functions to light computing applications.

The different plans available with their features can be found at the following url:
https://learn.microsoft.com/en-us/azure/azure-functions/functions-scale#service-limits

3) Create a static web page in the storage account

Storage accounts is the Azure object storage service (blobs). Its core functionality is to provide secure and highly available storage of stored objects, with pay-as-you-go based on the size of the stored data, and the accesses and writes to it. Uploaded files are organized into containers. You can configure the access of each object to allow access to anonymous users or not. If the access allows anonymous users, the access is public and a URL is assigned to each object, which will be accessible from the internet. Using this service, it is possible to set up a static website at a minimal cost.

We look for the Azure service called "Storage accounts"
Click on "Create"
Project Details:

- Subscription: Azure for Students
- Resource group: grupoest

Instance details:

- Storage Account Name: Web Storage
- Region: West Europe
- Performance: Azure blob storage
- Redundancy: Locally Redundant Storage (LRS)

  Redundancy defines the security of the data stored in the container. The redundancy type indicates how replicas of stored data are maintained. Options range from copies stored on different drives in the same data center (local redundancy) to copies stored in different geographies (geozone redundancy).

  For this application we don't need great security, that's why we chose storage with local redundancy, which is cheaper

- Click on "Next: Advanced Options"
  - Under "Security", we enable anonymous access in individual containers, and leave the rest of the options by default
- Click on "Next: Network"
  - Network connectivity
    - Network access: Enable public access from all networks
  - Network Routing
    - Routing Preference: Internet Routing
- Click on "Review and create"
- Click on "Create"

Within the storage account, we can create containers where we can store files.

Create a container within the newly created storage account:

- Open the newly created storage account

- In the left side menu, drop down "Data Storage" and select "Containers"
- Click on "+ Add container"
- New container:
    - Number: contenedorweb
    - Anonymous access tier: Blob (anonymous read access for blobs only)
- Click on "Create"

Enter the container that we have created and upload the files from the following website:
Save the following html code in a file called "index.html".

```
<! DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Button Example</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
        }
        button {
            padding: 10px 20px;
            font-size: 16px;
        }
    </style>
</head>
<body>
    <h1>Do NOT click the button:</h1>
    <button id="myButton">the button</button>
    <p id="output"></p>

    <script>
        // JavaScript to handle the button click event
        document.getElementById("myButton").addEventListener("click", function() {
            document.getElementById("output").textContent = "Are you happy now?";
        });
    </script>
</body>
</html>
```

Now upload the file inside the container:

- Click on "Upload"
- Drag the "index.html" file to the indicated box
- Click on "Upload"

We have now successfully uploaded the file to the storage container. Each "blob" file that is uploaded has a unique public url, if we have configured it to be publicly accessible. Let's look for the URL of the file:

- Click on the "index.html" file
- Under "Properties," copy the URL that appears

We paste the URL into the browser and observe the content of the page. The URL includes the name of the storage account, the name of the container, and the name of the file. However, this

way of accessing the page file only works if the page has a single file. It is possible to set up a static web page in the storage account, which will have its own container.

To create a static web page we open the storage account:

- In the left side menu we select "Static website"
- Under "Static website" we select "Enabled"
- The "Primary Endpoint" is the URL generated for the website
- In "Index document name" we write the document that is accessed when the URL is accessed without specifying a specific file
    - We wrote "index.html"
- "Error document path" indicates the document that is displayed if a resource that does not exist is accessed (error 404).
    - We leave it blank
- Click Save

A special container called $web has been created. The files of the website will be placed in this container.

- Access the container and upload the "index.html" file

Once you have uploaded the "index.html" file to the $web container, access the URL indicated in the "Main Endpoint" and check that the website is displayed correctly.